

GraphAnchor Codebase Explanation

GraphAnchor is a lightweight, local-first Graph RAG (Retrieval-Augmented Generation) backend. It works by ingesting raw text, chunking it, extracting semantic entities and relationships to form a Knowledge Graph, and then querying that graph in combination with vector search.

Below is a detailed, folder-by-folder explanation of the codebase structure and what each module does.

1. Project Root Directory

The root directory holds the main application entry point and configuration files for the environment, containerization, and dependencies.

- **main.py**: The core FastAPI application. It defines all the API routes (e.g., `/ingest`, `/query`, `/answer`, `/reset`, and `/graph/stats`). It ties together the different modules from `src/` to handle a complete workflow: accepting a file upload, passing it to the ingestion and storage modules, and answering queries by combining vector retrieval with graph traversal. It also serves a simple frontend (`web/index.html`).
 - **pyproject.toml & uv.lock**: Defines the Python project dependencies (like `fastapi`, `ollama`, `chromadb`, etc.) and metadata. The `uv.lock` file ensures that the exact versions of dependencies are locked for reproducible builds.
 - **Dockerfile & docker-compose.yml**: Used for containerizing the application. The `docker-compose.yml` file sets up the FastAPI backend alongside a local `ollama` container, ensuring the required local LLM models (e.g., `qwen2.5-coder:7b`) are automatically pulled and available.
 - **README.md**: The main documentation for the project, detailing setup instructions, features, and how to test the application.
-

2. The src/ Directory

The `src/` directory contains all the modular logic of the Graph RAG engine, separated into distinct responsibilities.

config.py

This file handles the configuration settings for the application. It defines an `AppConfig` class using `Pydantic`, which sets defaults for things like the LLM models to use, chunk size, overlapping logic, and file size limits. It attempts to load overrides from `config.yaml` if it exists.

ingestion.py

This module is responsible for reading incoming data and preparing it for the AI pipeline: - **Text Extraction**: Uses libraries like `pypdf` to parse PDFs, or standard decoding for `.txt/.md` files, ensuring clean UTF-8 text is returned. - **Token Chunking**: Uses OpenAI's `tiktoken` to split the long extracted text into manageable, overlapping chunks. Overlapping ensures that context isn't lost if a sentence is split across two chunks.

storage.py

This module manages the dual-storage architecture of the system: - **SQLite Database**: Handles robust tabular data. It stores the raw `documents`, the individual `text chunks`, and the `graph edges` (relationships between entities). It handles foreign key relationships (e.g., an edge belongs to a chunk, a chunk belongs to a document). - **ChromaDB**: The vector database. It stores the mathematical embeddings of chunks and entities. When you search for a query, ChromaDB finds the chunks mathematically closest in meaning to your query.

retrieval.py

This handles the vector embedding pipeline. It connects to the local Ollama instance to turn raw text strings into dense vector representations (e.g., using the `snowflake-arctic-embed2` model). It features automatic retries in case the Ollama service is temporarily busy.

generation.py

The "brain" of the extraction and answering process: - **Graph Extraction**

(extract_graph_from_chunk): Given a text chunk, it prompts the local LLM in strict JSON mode to extract semantic triples (Entity -> Relation -> Target). - **Answer Synthesis (generate_answer / stream_answer)**: After relevant chunks and graph edges are retrieved, this module builds a highly specific prompt instructing the LLM to provide an objective, factual answer using *only* the retrieved evidence. It supports both complete responses and real-time streaming (token-by-token).

3. The tests/ Directory

Contains the automated test suite ensuring reliability: - **test_api.py**: Uses `pytest` and FastAPI's `TestClient` to programmatically test the system. It simulates users uploading files, running vector searches (Medium difficulty), and performing complex graph traversal queries (Hard difficulty) to ensure the logic works flawlessly without human intervention. The tests mock the LLM where appropriate for rapid, reliable CI/CD pipelines.

4. The web/ & data/ Directories

- **web/index.html**: A minimalistic, single-page frontend application. It interacts with the backend's API endpoints via JavaScript to allow users to visually upload files and ask questions, optionally drawing the graph nodes in the browser.
 - **data/**: The local directory where SQLite (`graph.db`) and ChromaDB persist their database files on your hard drive, keeping everything 100% local.
-

Conclusion

GraphAnchor is a tightly integrated system. A request starts in `main.py`, text is processed via `ingestion.py`, graphs are intelligently mapped by `generation.py`, embedded by `retrieval.py`, saved by `storage.py`, and rigorously validated by the `tests/` suite. All components work together locally and securely.